

Lecture 10: Finite State representation of digital circuits

We noted that any member of the class of circuits called flip flops can be conveniently be represented by a finite state machine, and in general any synchronous circuit can be modelled by a finite state machine, and *vice versa*. A finite state machine in its most general form can be represented by a pair of equations:

$$\begin{aligned}S(t+1) &= f(S(t), I(t)) \\ O(t+1) &= g(S(t), I(t))\end{aligned}$$

Where $S(t)$ represents the state at time t and can take only a finite set of values, normally thought of as integers. $I(t)$ and $O(t)$ are the inputs and outputs which are also normally discrete bounded variables and f and g are functions. Synchronous circuits which conform to the above equations are called Mealy machines, and are represented by the block diagram of Figure 1.

There is a simpler form of finite state machine which is often used where the output is only a function of the state, and the equation pair becomes:

$$\begin{aligned}S(t+1) &= f(S(t), I(t)) \\ O(t) &= g(S(t))\end{aligned}$$

Circuits of this form are called Moore machines, and are represented by the block diagram of Figure 2. In both

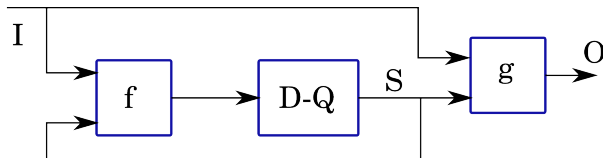


Figure 1: The Mealy Machine

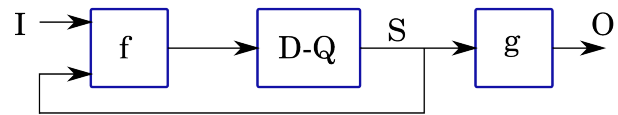


Figure 2: The Moore Machine

cases the blocks f and g compute the finite state machine functions and are implemented by combinational logic. We will call block g the output (or decode) logic and block f the state sequencing (or input) logic. The blocks marked $D - Q$ are called state registers and consist of a set of D-type flip flops.

The fact that synchronous circuits can be represented by finite state machines gives us the basis of a design methodology. As with programme design (or indeed any engineering design task) we can most easily solve a problem at the highest functional level, hence it is better to design the finite state machine before considering how to construct it out of gates. The methodology for synchronous circuit design can be stated as follows:

1. Determine the number of states required by the system
2. Determine the state transitions and outputs and draw the finite state machine
3. Choose the way in which the states will be represented (State Assignment)
4. Express the state sequencing logic as a set of Boolean equations combining the states and the inputs, using the existing methodology based on finding minterms. Minimise using the Karnaugh map method.
5. Express the required outputs as a Boolean function of the states, and minimize using Karnaugh maps if possible.
6. Draw the circuit.

We will now consider a simple example which will illustrate some of the issues concerned with this methodology. It is a counting circuit, similar to that designed in the previous lecture. The input will be taken from a push button, and the output will be a digit in the range 0 to 5 displayed on a seven segment display (see Figure 3). The seven segment display device will be set up so that a logic 1 makes the corresponding segment visible, and a logic 0 will make it invisible. Such a circuit might used to set the time in a digital clock.

The design of the finite state machine is quite straightforward. Clearly there are six states, which we can label 0 to 5 representing the digit that will be displayed. If we assume that the circuit operates on a slow clock, providing a falling edge once every second, and the display advances when the input button is pushed ($I = 1$)

then the state transitions are straightforward. The outputs pose more of a problem since we must decide which of the seven segments to be visible for each state. For example in state 0 we require segments 0,1,2,4,5 and 6 to be visible. Thus we might write the corresponding output A as 1,1,1,0,1,1. When we have specified all the outputs we have completed the finite state machine design, which is given in Figure 4. Note the convention that in a Moore machine the outputs are written in the nodes, (below the state name). In the Mealy machine they are written on the arcs. In order to translate a finite state machine into a circuit, we must first decide how the states

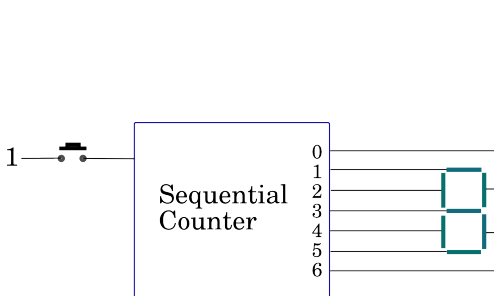


Figure 3: A Sequential Design Problem

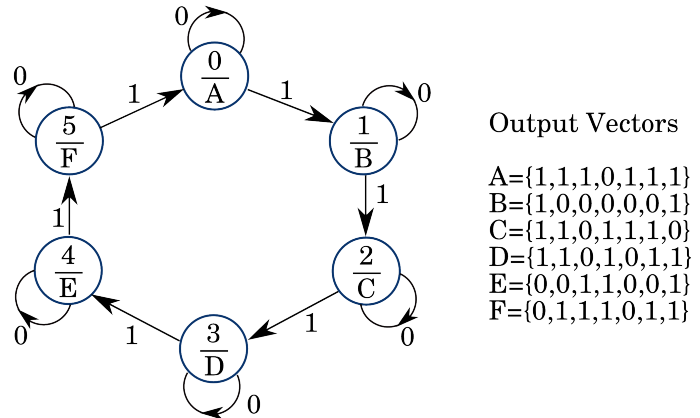


Figure 4: A Moore Machine of the six state counter

are to be represented. The simplest possibility is that we allocate one flip flop per state, so that our D-Q box in Figure 2 is made up of six flip flops. However, this arrangement is likely to use many more components than we need, since the states could be encoded on just three flip flops rather than six. But here again is a problem, since three flip flops can represent eight states, that is to say, if we treat the outputs of the flip flops as a three bit number, we have eight possibilities, and we need to decide which number encodes (or represents) each state. This is the state assignment problem, and the choices that we make will affect both the output and the state sequencing logic.

We will choose to make the state allocation simple. State 0 will be allocated the three bit binary number ($Q_2Q_1Q_0$) 000, state 1 will be allocated 001 and so on. If we use a binary variable S_i to represents the i^{th} state, we see immediately that each state has a corresponding minterm with the three flip flop outputs as its variables, and we can define the assignment using these minterms:

$$S_0 = Q_2' \cdot Q_1' \cdot Q_0' \quad S_1 = Q_2' \cdot Q_1' \cdot Q_0 \quad \text{etc}$$

Only one of the eight states S_i will evaluate to logic 1 at any one time. The output logic is very simple in this case. It is specified entirely by the OR function. For example, consider output 1 (which makes the top segment visible). It should be one only when we are displaying the numbers 0,2,3 and 5. That is to say, it is a one when we are in states S_0 , S_2 , S_3 and S_5 . Thus we can write:

$$O_1 = S_0 + S_2 + S_3 + S_5$$

$$O_1 = Q_2' \cdot Q_1' \cdot Q_0' + Q_2' \cdot Q_1 \cdot Q_0' + Q_2' \cdot Q_1 \cdot Q_0 + Q_2 \cdot Q_1' \cdot Q_0$$

and apply the Karnaugh map method to determine the minimum circuit for O_1 . The same method is used for all the other output lines. We can now construct the input logic by considering each state in turn. The design decomposes into a separate problem for each state. The equations for a state can be derived simply by looking at the arrows that point to that state in the finite state machine diagram. We can obtain an equation directly in the canonical form. For example:

$$N_0 = S_5 \cdot I + S_0 \cdot I'$$

where N_0 means the next state is to be S_0 . Here again we have a problem that we can minimise using the standard Karnaugh map method. However, the new state needs to be presented to the D input of the flip flops, and thus must be encoded. For example if the next state is to be 1, we need to set the D inputs to be $D_2 = 0$, $D_1 = 0$ and $D_0 = 1$. In practice we formulate the equations the other way round, saying that the D_0 value should be set to 1 when the next state is S_1 (001) or S_3 (011) or S_5 (101). D_0 is the bottom bit of the number that defines the state. Written as a Boolean equation this is:

$$D_0 = N_1 + N_3 + N_5$$

similarly:

$$D_1 = N_2 + N_3 \text{ and } D_2 = N_4 + N_5$$

We can substitute for the N values to obtain for example

$$D_1 = S_1 \cdot I + S_2 \cdot I' + S_2 \cdot I + S_3 \cdot I'$$

$$D_1 = Q'_2 \cdot Q'_1 \cdot Q_0 \cdot I + Q'_2 \cdot Q_1 \cdot Q'_0 \cdot I' + Q'_2 \cdot Q_1 \cdot Q'_0 \cdot I + Q'_2 \cdot Q_1 \cdot Q_0 \cdot I'$$

Following the same procedure for D_0 and D_2 we obtain the Boolean equations of the whole state sequencing logic in canonical form ready for minimisation. Once the state assignment has been made, the unused states become don't care states, and we expect that they will never appear when the circuit is in correct operation. Remembering that don't cares may be treated as either zero or one, and, when using the minterm canonical form we can often benefit if we treat some of them as ones. We add them to the Karnaugh maps for simplification of the resulting circuit. When a synchronous circuit has been designed in this way it is necessary to check

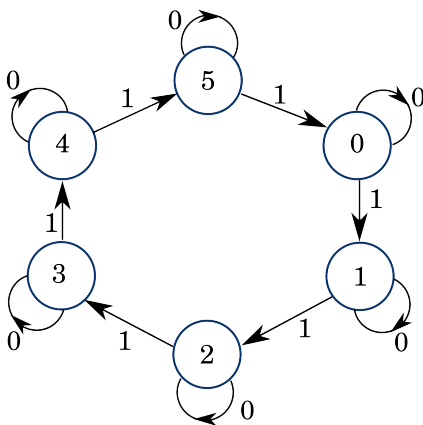


Figure 5: Unallocated States

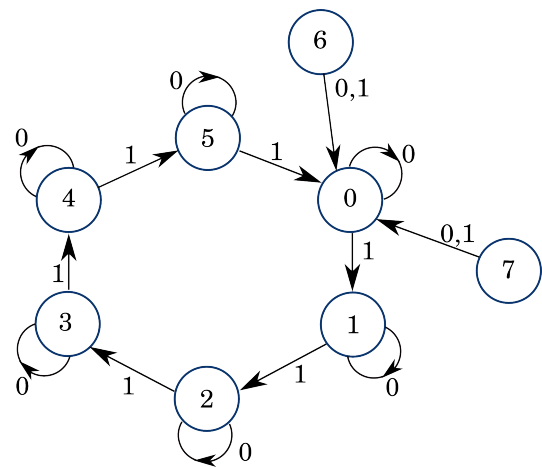


Figure 6: Explicitly Allocated States

that it will start up correctly. The problem is that, since we have treated the unassigned states as don't cares, we cannot define how they will behave. For example, it is possible that the unassigned states form a closed finite state machine as shown in Figure 5, and on start up the circuit never enters any of the required states and therefore does not perform correctly. We could avoid this problem by explicitly including the unassigned states in the design, for example making them all lead to a known state, as shown in Figure 6, but this would have the disadvantage that the resulting circuit may not be the smallest possible. Another strategy would be to check to see what state transitions are implied when the don't cares have been allocated particular values, and modify the circuit if it is unsafe. A third possibility is to use the set and reset features on standard flip flops to force the circuit into an assigned state either on start up, or on the press of a reset button.

Unfortunately, in general, there is no easy way to see which state assignment will result in the best simplification of the whole circuit. One strategy is to try out all possibilities, and determine which gives the best overall result. At first sight this looks like a daunting task, since the number of ways we could allocate the eight possibilities to the six states is 56. However, fortunately many of them can be eliminated. For example, consider one possible state assignment shown in the table below.

State	Assignment	Isomorphs
0	000	100 010 110
1	001	101 011 111
2	010	110 000 100
3	011	111 001 101
4	100	000 110 010
5	101	001 111 011

We note that flip flops almost always have complementary outputs, and thus if we negate any column this will not result in a different circuit, since all we do is exchange the Q and Q' outputs of the flip flop. Applying this principle we can generate a further seven assignments, which are isomorphic to our original one, by negating different combinations of columns. Three are shown in the table. The first is created by negating column 1, the second by negating column 2, and the third by negating columns 1 and 2.

A small refinement of our specification is to make the digit change on the 0 to 1 transition of the input button. We could do this by buffering the input through a T type flip flop, but an alternative strategy is to use the design of Figure 7. We have here a twelve state machine, so the strategy of using one flip flop per state will be very wasteful. We can choose to use four flip flops, and make an assignment of the sixteen possibilities to the twelve states, but now the number of possible assignments increases to 1820, and even allowing for the symmetries we have over a hundred possibilities to choose from. Thus to determine the optimal state assignment it will not be possible to evaluate all possibilities. Consequently we need to rely on heuristic rules of the form:

1. All those states that have the same next state for the same input should be given adjacent state assignments
2. The next states of a state produced by applying adjacent input conditions should be given adjacent state assignments.

In our case the second rule suggests that the neighbouring states in our diagram should be given adjacent state assignments, ie going round the ring in Figure 4 we would use 000 001 011 111 110 100 and back to 000. There are many such rules quoted but it is beyond the scope of this course to discuss them.

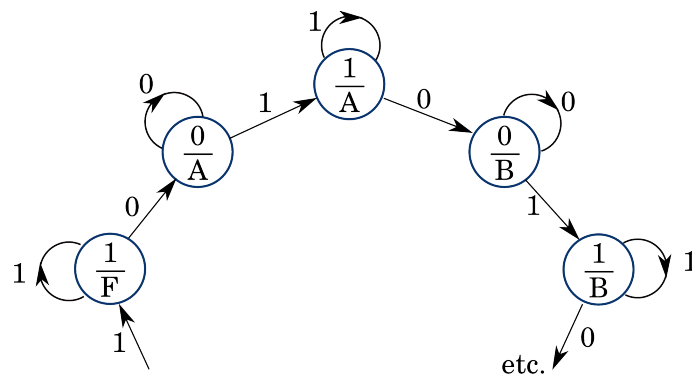


Figure 7: An Edge Triggered Counter